01-02-python-intro

January 7, 2026

```
<div style='float:left;'><h1>CPSC 4300/6300: Applied Data Science</h1></div>
<img style="float: right; padding-right: 10px; width: 65px" src="https://raw.githubusercontent.c
```

0.1 Week 1 | Lab : Introduction to Python and its Numerical Stack

Clemson University Instructor(s): Tim Ransom

Learning goals

- Write basic Python code using functions, loops, and data structures.
- Compare and contrast Python lists and NumPy arrays.
- Utilize statistical libraries like scipy.stats and statsmodels.

0.2 Programming Expectations

All lab assignments and Homework sets for this class will use Python and the browser-based iPython notebook format you are currently viewing. Programming at the level of CPSC 2120 is a prerequisite for this course. If you have concerns about this, come speak with any of the instructors.

We will refer to the Python 3 documentation in this lab and throughout the course.

0.3 About

- This introductory lab is a condensed introduction to Python numerical programming.

- By the end of this lab, you will feel more comfortable:
  1. Writing short Python code using functions, loops, lists, numpy arrays, and dictionaries.
  2. Manipulating Python lists and numpy arrays and understanding the difference between them.

[1]:
```
""" RUN THIS CELL TO GET THE RIGHT FORMATTING """
import requests
from IPython.core.display import HTML
css_file = 'https://raw.githubusercontent.com/bsethwalker/clemson-cs4300/main/
 ↪css/cpsc6300.css'
styles = requests.get(css_file).text
HTML(styles)
```

[1]:
```
<IPython.core.display.HTML object>
```

1 1 - Getting Started with Python and it's numerical stack

1.0.1 1.1 Importing modules

All notebooks should begin with code that imports *modules*, collections of built-in, commonly-used Python functions. Below we import the Numpy module, a fast numerical programming library for scientific computing. Future labs will require additional modules, which we'll import with the same syntax.

```
import MODULE_NAME as MODULE_NICKNAME
```

[2]:
```
import numpy as np
```

Now that Numpy has been imported, we can access some useful functions. For example, we can use `mean` to calculate the mean of a set of numbers.

[3]:
```
my_list = [1.2, 2, 3.3]

np.mean(my_list)
```

[3]:
```
2.1666666666666665
```

1.0.2 1.2 Calculations and variables

[4]:
```
# // is integer division
print("Hello")
1/2, 1//2, 1.0/2, 3*3.2
```

```
Hello
```

[4]:
```
(0.5, 0, 0.5, 9.600000000000001)
```

The last line in a cell is returned as the output value, as above. For cells with multiple lines of results, we can display results using `print`, as can be seen below.

[5]:
```
print(1 + 3.0, "\n", 9, 7)
5/3
```

```
4.0
 9 7
```

[5]:
```
1.6666666666666667
```

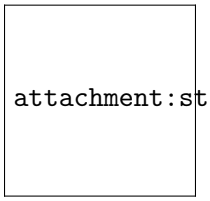
We can store integer or floating point values as variables. The other basic Python data types – booleans, strings, lists – can also be stored as variables.

[6]:
```
a = 1
b = 2.0
```

Here is the storing of a list

[7]:
```
a = [1, 2, 3]
```

Think of a variable as a label for a value, not a box in which you put the value



(**Image:** Fluent Python by Luciano Ramalho)

[ ]:
```
b = a
b
```

This DOES NOT create a new copy of `a`. It merely puts a new label on the memory at a, as can be seen by the following code:

[8]:
```
print("a", a)
print("b", b)
a[1] = 7
print("a after change", a)
print("b after change", b)
```

```
a [1, 2, 3]
b 2.0
a after change [1, 7, 3]
b after change 2.0
```

1.0.3 1.3 Tuples

Multiple items on one line in the interface are returned as a *tuple*, an immutable sequence of Python objects. See the end of this notebook for an interesting use of `tuples`.

[10]:
```
a = 1
b = 2.0
a + a, a - b, b * b, 10*a
```

[10]: `(2, -1.0, 4.0, 10)`

`type()` We can obtain the type of a variable, and use boolean comparisons to test these types. VERY USEFUL when things go wrong and you cannot understand why this method does not work on a specific variable!

[11]:
```
type(a) == float
```

[11]: `False`

[12]:
```
type(a) == int
```

[12]: `True`

[13]:
```
type(a)
```

[13]: `int`

For reference, below are common arithmetic and comparison operations.

attachment:ops1_v2.png

attachment:ops2_v2.png

EXERCISE 1: Create a tuple called `tup` with the following seven objects:

- The first element `a` is an integer of your choice
- The second element `b` is a float of your choice

- The third element `c` is the sum of the first two elements
- The fourth element `d` is the difference of the first two elements

- The fifth element `e` is the first element divided by the second element
- Create a tuple called `tup` and add all above elements to it.
- Display the output of `tup`. What is the type of the variable `tup`? What happens if you try and chage an item in the tuple?

[16]:
```
"""Your code for exercise 1 here:"""

# your code here
a = 16
b = 5.0
c = a + b
d = a - b
e = a / b

tup = (a, b, c, d, e)

print(tup)
print(type(tup))
```

```
(16, 5.0, 21.0, 11.0, 3.2)
<class 'tuple'>
```

[ ]:
```

```

[17]:
```
# this is supposed to throw an error so you can see what an error looks like in␣
 ↪python notebooks (this is not graded)
try:
    tup[2] = 3
except NameError:
    print('Tuple object does not support item assignment')
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[17], line 3
      1 # this is supposed to throw an error so you can see what an error looks␣
 ↪like in python notebooks (this is not graded)
      2 try:
----> 3     tup[2] = 3
      4 except NameError:
      5     print('Tuple object does not support item assignment')

TypeError: 'tuple' object does not support item assignment
```

1.0.4 1.4 Lists

Much of Python is based on the notion of a list. In Python, a list is a sequence of items separated by commas, all within square brackets. The items can be integers, floating points, or another type.

Unlike in C arrays, items in a Python list can be different types, so Python lists are more versatile than traditional arrays in C or other languages.

Let's start out by creating a few lists.

[18]:
```
empty_list = []
float_list = [1., 3., 5., 4., 2.]
int_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
mixed_list = [1, 2., 3, 4., 5]

print(empty_list)
print(int_list)
print(mixed_list, float_list)
```

```
[]
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[1, 2.0, 3, 4.0, 5] [1.0, 3.0, 5.0, 4.0, 2.0]
```

Lists in Python are zero-indexed, as in C. The first entry of the list has index 0, the second has index 1, and so on.

[19]:
```
print(int_list[0])
print(float_list[1])
```

```
1
3.0
```

What happens if we try to use an index that doesn't exist for that list? Python will complain!

[20]:
```
try:
    print(float_list[10])
except IndexError:
    print("list index out of range")
```

```
list index out of range
```

You can find the length of a list using the built-in function `len`:

[21]:
```
print(float_list)
len(float_list)
```

```
[1.0, 3.0, 5.0, 4.0, 2.0]
```

[21]: 5

1.0.5 1.5 Indexing on lists plus Slicing

And since Python is zero-indexed, the last element of `float_list` is

[22]:
```
float_list[len(float_list)-1]
```

[22]: 2.0

It is more idiomatic in Python to use -1 for the last element, -2 for the second last, and so on

[23]:
```
float_list[-3]
```

[23]:
```
5.0
```

We can use the : operator to access a subset of the list. This is called **slicing.**

[24]:
```
print(float_list[1:-1])
print(float_list[0:2])
```

```
[3.0, 5.0, 4.0]
[1.0, 3.0]
```

[ ]:
```
lst = ['hi', 7, 'c', 'cat', 'hello', 8]
```

Below is a summary of list slicing operations: attachment:ops3_v2.png

[25]:
```
lst = ['hi', 7, 'c', 'cat', 'hello', 8]
lst[:2]
```

[25]:
```
['hi', 7]
```

You can slice "backwards" as well:

[26]:
```
float_list[:-2] # up to second last
```

[26]:
```
[1.0, 3.0, 5.0]
```

[28]:
```
float_list[:4] # up to but not including 5th element
```

[28]:
```
[1.0, 3.0, 5.0, 4.0]
```

You can also slice with a stride:

[29]:
```
float_list[:4:2] # above but skipping every second element
```

[29]:
```
[1.0, 5.0]
```

We can iterate through a list using a loop. Here's a **for loop.**

[30]:
```
for ele in float_list:
    print(ele)
```

```
1.0
3.0
5.0
```

```
4.0
2.0
```

What if you wanted the index as well?

Use the built-in python method `enumerate`, which can be used to create a list of tuples with each tuple of the form `(index, value)`.

[31]:
```
for i, ele in enumerate(float_list):
    print(i, ele)
```

```
0 1.0
1 3.0
2 5.0
3 4.0
4 2.0
```

1.0.6 1.6 Appending and deleting

We can also append items to the end of the list using the `+` operator or with `append`.

[38]:
```
new_list = float_list + [.333]
new_list
```

[38]:
```
[1.0, 3.0, 4.0, 2.0, 0.444, 0.444, 0.333]
```

[39]:
```
float_list.append(.444)
```

[40]:
```
print(float_list)
len(float_list)
```

```
[1.0, 3.0, 4.0, 2.0, 0.444, 0.444, 0.444]
```

[40]:
```
7
```

Now, run the cell with `float_list.append()` a second time. Then run the subsequent cell. What happens?

To remove an item from the list, use `del`.

[41]:
```
del(float_list[2])
print(float_list)
```

```
[1.0, 3.0, 2.0, 0.444, 0.444, 0.444]
```

You may also add an element (elem) in a specific position (index) in the list

[42]:
```
elem = '3.14'
index = 1
float_list.insert(index, elem)
float_list
```

[42]: [1.0, '3.14', 3.0, 2.0, 0.444, 0.444, 0.444]

1.0.7 1.7 List Comprehensions

Lists can be constructed in a compact way using a *list comprehension*. Here's a simple example.

[43]:
```
int_list
```

[43]: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

[44]:
```
squaredlist = []

for i in int_list:
    squaredlist.append(i*i)
squaredlist
```

[44]: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

[45]:
```
squaredlist = [i*i for i in int_list]
squaredlist
```

[45]: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

And here's a more complicated one, requiring a conditional.

[46]:
```
comp_list1 = [2*i for i in squaredlist if i % 2 == 0]
print(comp_list1)
```

```
[8, 32, 72, 128, 200]
```

This is entirely equivalent to creating `comp_list1` using a loop with a conditional, as below:

[47]:
```
comp_list2 = []
for i in squaredlist:
    if i % 2 == 0:
        comp_list2.append(2*i)

print(comp_list2)
```

```
[8, 32, 72, 128, 200]
```

The list comprehension syntax

```
[expression for item in list if conditional]
```

is equivalent to the syntax

```
for item in list:
    if conditional:
        expression
```

Exercise 2.1: Build a list called "primes" that contains every prime number between 1 and 100, in two different ways:

Using `for` loop and conditional `if` statements.

- Build a list named `primes` that contains every prime number between 1 and 100
- **Note:** Complete above task by only using `for` loops and conditional `if` statements.

[58]:
```
"""Your code for exercise 2.1 here:"""

# your code here

primes = []

for num in range(2, 101):
    is_prime = True

    for i in range(2, num):
        if num % i == 0:
            is_prime = False
            break

    if is_prime:
        primes.append(num)

print(primes)
```

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
79, 83, 89, 97]
```

[ ]:
```

```

Exercise 2.2: Build a list called "primes" that contains every prime number between 1 and 100, in two different ways:

(Stretch Goal) Using a list comprehension. You should be able to do this in one line of code.

- Build a list named `primes` that contains every prime number between 1 and 100
- **Complete above task using list comprehension.**

Hint: It might help to look up the function `all()` in this documentation.

[66]:
```
"""Your code for exercise 2.2 here:"""

# your code here
primes = [n for n in range(2, 101) if all(n % i for i in range(2, int(n**0.5) +␣
↪1))]

print(primes)
```

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
79, 83, 89, 97]
```

[ ]:

1.0.8 1.8 Functions

A *function* object is a reusable block of code that does a specific task. Functions are commonplace in Python, either on their own or as they belong to other objects. To invoke a function `func`, you call it as `func(arguments)`.

We've seen built-in Python functions and methods (details below). For example, `len()` and `print()` are built-in Python functions. And at the beginning, you called `np.mean()` to calculate the mean of three numbers, where `mean()` is a function in the numpy module and numpy was abbreviated as `np`. This syntax allows us to have multiple "mean" functions in different modules; calling this one as `np.mean()` guarantees that we will execute numpy's mean function, as opposed to a mean function from a different module.

1.8.1 User-defined functions We'll now learn to write our own user-defined functions. Below is the syntax for defining a basic function with one input argument and one output. You can also define functions with no input or output arguments, or multiple input or output arguments.

```
def name_of_function(arg):
    ...
    return(output)
```

We can write functions with one input and one output argument. Here are two such functions.

[63]:
```
def square(x):
    x_sqr = x*x
    return(x_sqr)

def cube(x):
    x_cub = x*x*x
    return(x_cub)

square(5),cube(5)
```

[63]:
```
(25, 125)
```

What if you want to return two variables at a time? The usual way is to return a tuple:

[64]:
```
def square_and_cube(x):
    x_cub = x*x*x
    x_sqr = x*x
    return(x_sqr, x_cub)

square_and_cube(5)
```

[64]:
```
(25, 125)
```

1.8.2 Lambda functions Often we quickly define mathematical functions with a one-line function called a *lambda* function. Lambda functions are great because they enable us to write functions without having to name them, ie, they're *anonymous*.
No return statement is needed.

[65]:
```
# create an anonymous function and assign it to the variable square
square = lambda x: x*x
print(square(3))

hypotenuse = lambda x, y: x*x + y*y

## Same as
# def hypotenuse(x, y):
#     return(x*x + y*y)

hypotenuse(3,4)
```

```
9
```

[65]: 25

1.0.9 1.9 Methods

A function that belongs to an object is called a *method*. By "object," we mean an "instance" of a class (e.g., list, integer, or floating point variable).

For example, when we invoke `append()` on an existing list, `append()` is a method.

In other words, a *method* is a function on a specific *instance* of a class (i.e., *object*). In this example, our class is a list. `float_list` is an instance of a list (thus, an object), and the `append()` function is technically a *method* since it pertains to the specific instance `float_list`.

[ ]:
```
float_list = [1.0, 2.09, 4.0, 2.0, 0.444]
print(float_list)
float_list.append(56.7)
float_list
```

Exercise 3: generate a list of the prime numbers between 1 and 100

- In Exercise 2, above, you wrote code that generated a list of the prime numbers between 1 and 100.
- Now, write a function called `isprime()` that takes in a positive integer N, and determines whether or not it is prime.

- Return `True` if it's prime and return `False` if it isn't.
- Then, using a list comprehension and `isprime()`, create a list `myprimes` that contains all the prime numbers less than 100.

[136]:
```
"""Your code for exercise 3 here:"""

# your code here
```

```
def isprime(N):

    if N < 2:
        return False

    for i in range(2, int(N**0.5) + 1):
        if N % i == 0:
            return False

    return True


myprimes = [num for num in range(1, 100) if isprime(num)]

print(myprimes)
```

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
79, 83, 89, 97]
```

[ ]:

1.0.10 1.10 Introduction to Numpy

Scientific Python code uses a fast array structure, called the numpy array. Those who have programmed in Matlab will find this very natural. For reference, the numpy documention can be found here.

Let's make a numpy array:

[87]:
```
my_array = np.array([1, 2, 3, 4])
my_array
```

[87]:
```
array([1, 2, 3, 4])
```

[88]:
```
# works as it would with a standard list
len(my_array)
```

[88]:
```
4
```

The shape array of an array is very useful (we'll see more of it later when we talk about 2D arrays – matrices – and higher-dimensional arrays).

[89]:
```
my_array.shape
```

[89]:
```
(4,)
```

Numpy arrays are **typed**. This means that by default, all the elements will be assumed to be of the same type (e.g., integer, float, String).

[90]:
```
my_array.dtype
```

[90]:
```
dtype('int64')
```

Numpy arrays have similar functionality as lists! Below, we compute the length, slice the array, and iterate through it (one could identically perform the same with a list).

[91]:
```
print(len(my_array))
print(my_array[2:4])

for ele in my_array:
    print(ele)
```

```
4
[3 4]
1
2
3
4
```

There are two ways to manipulate numpy arrays a) by using the numpy module's methods (e.g., `np.mean()`) or b) by applying the function np.mean() with the numpy array as an argument.

[92]:
```
print(my_array.mean())
print(np.mean(my_array))
```

```
2.5
2.5
```

A `constructor` is a general programming term that refers to the mechanism for creating a new object (e.g., list, array, String).

There are many other efficient ways to construct numpy arrays. Here are some commonly used numpy array constructors. Read more details in the numpy documentation.

[93]:
```
np.ones(10) # generates 10 floating point ones
```

[93]:
```
array([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
```

Numpy gains a lot of its efficiency from being typed. That is, all elements in the array have the same type, such as integer or floating point. The default type, as can be seen above, is a float. (Each float uses either 32 or 64 bits of memory, depending on if the code is running a 32-bit or 64-bit machine, respectively).

[94]:
```
np.dtype(float).itemsize # in bytes (remember, 1 byte = 8 bits)
```

[94]:
```
8
```

[95]:
```
np.ones(10, dtype='int') # generates 10 integer ones
```

[95]:
```
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
```

[96]:
```
np.zeros(10)
```

[96]:
```
array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

Often, you will want random numbers. Use the `random` constructor!

[97]:
```
np.random.random(10) # uniform from [0,1]
```

[97]:
```
array([0.5290328 , 0.48688281, 0.7196986 , 0.88014752, 0.9374566 ,
       0.25053371, 0.76687635, 0.56783345, 0.83754641, 0.28255381])
```

You can generate random numbers from a normal distribution with mean 0 and variance 1:

[98]:
```
normal_array = np.random.randn(1000)
print("The sample mean and standard devation are %f and %f, respectively." %(np.
 ↪mean(normal_array), np.std(normal_array)))
```

```
The sample mean and standard devation are 0.038778 and 0.975102, respectively.
```

[99]:
```
len(normal_array)
```

[99]:
```
1000
```

You can sample with and without replacement from an array. Let's first construct a list with evenly-spaced values:

[100]:
```
grid = np.arange(0., 1.01, 0.1)
grid
```

[100]:
```
array([0. , 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1. ])
```

Without replacement

[103]:
```
np.random.choice(grid, 5, replace=False)
```

[103]:
```
array([0.8, 0.7, 0.5, 1. , 0.6])
```

[104]:
```
# supposed to error out - read the error message!
np.random.choice(grid, 20, replace=False)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[104], line 2
      1 # supposed to error out - read the error message!
----> 2 np.random.choice(grid, 20, replace=False)

File mtrand.pyx:1000, in numpy.random.mtrand.RandomState.choice()

ValueError: Cannot take a larger sample than population when 'replace=False'
```

With replacement:

[105]:
```
np.random.choice(grid, 20, replace=True)
```

[105]:
```
array([0.1, 0.3, 0.6, 0.2, 0.6, 0.9, 0.4, 0.7, 0.8, 0.1, 0.8, 0.6, 0.9,
       1. , 0.3, 0.6, 0.4, 0. , 0.6, 0.4])
```

1.0.11 1.11 Tensors

We can think of tensors as a name to include multidimensional arrays of numerical values. While tensors first emerged in the 20th century, they have since been applied to numerous other disciplines, including machine learning. In this class you will only be using **scalars**, **vectors**, and **2D arrays**, so you do not need to worry about the name 'tensor'.

We will use the following naming conventions:

- scalar = just a number = rank 0 tensor ($a \quad F$,)
- vector = 1D array = rank 1 tensor ($x = (\ x_1, ..., x_i\ ) \quad F^n$)
- matrix = 2D array = rank 2 tensor ($\mathbf{X} = [a_{ij}]F^{mn}$)
- 3D array = rank 3 tensor ($\mathscr{X} = [t_{i,j,k}]F^{mnl}$)
- $\mathscr{N}$D array = rank $\mathscr{N}$ tensor ($\mathscr{T} = [t_{i1}, ..., t_{i\mathscr{N}}]F^{n_1 \ldots n_{\mathscr{N}}}$)

1.0.12 Slicing a 2D array

attachment:slicing_2D_oreilly.png

source: oreilly

[106]:
```
# how do we get just the second row of the above array?
```

Numpy supports vector operations What does this mean? It means that instead of adding two arrays, element by element, you can just say: add the two arrays.

[107]:
```
first = np.ones(5)
second = np.ones(5)
first + second # adds in-place
```

[107]:
```
array([2., 2., 2., 2., 2.])
```

Note that this behavior is very different from python lists where concatenation happens.

[108]:
```
first_list = [1., 1., 1., 1., 1.]
second_list = [1., 1., 1., 1., 1.]
first_list + second_list # concatenation
```

[108]:
```
[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
```

On some computer chips, this numpy addition actually happens in parallel and can yield significant increases in speed. But even on regular chips, the advantage of greater readability is important.

Broadcasting Numpy supports a concept known as *broadcasting*, which dictates how arrays of different sizes are combined together. There are too many rules to list here, but importantly, multiplying an array by a number multiplies each element by the number. Adding a number adds the number to each element.

[109]:
```
first_list = np.array([1., 1., 1., 1., 1.])
first_list + 1
```

[109]:
```
array([2., 2., 2., 2., 2.])
```

[110]:
```
first*5
```

[110]:
```
array([5., 5., 5., 5., 5.])
```

This means that if you wanted the distribution $N(5, 7)$ you could do:

[111]:
```
normal_5_7 = 5 + 7*normal_array
np.mean(normal_5_7), np.std(normal_5_7)
```

[111]:
```
(5.27144275526082, 6.825713096852473)
```

Multiplying two arrays multiplies them element-by-element

[112]:
```
(first +1) * (first*5)
```

[112]:
```
array([10., 10., 10., 10., 10.])
```

You might have wanted to compute the dot product instead:

[113]:
```
np.dot((first +1) , (first*5))
```

[113]:
```
50.0
```

1.0.13 1.12 Probabilitiy Distributions from `scipy.stats` and `statsmodels`

Two useful statistics libraries in python are `scipy` and `statsmodels`.

For example to load the z_test: (A two-tailed test is appropriate if you want to determine if there is any difference between the groups you are comparing. For instance, if you want to see if Group A scored higher or lower than Group B, then you would want to use a two-tailed test.)

[114]:
```
import statsmodels
from statsmodels.stats.proportion import proportions_ztest
```

[115]:
```
x = np.array([74,100])
n = np.array([152,266])
```

```
zstat, pvalue = statsmodels.stats.proportion.proportions_ztest(x, n)
print("Two-sided z-test for proportions: \n","z =",zstat,", pvalue =",pvalue)
```

```
Two-sided z-test for proportions:
 z = 2.212695207500177 , pvalue = 0.026918666032452288
```

[116]:
```
#The `%matplotlib inline` ensures that plots are rendered inline in the browser.
%matplotlib inline
import matplotlib.pyplot as plt
```

Let's get the normal distribution namespace from `scipy.stats`. See here for Documentation.

[117]:
```
from scipy.stats import norm
```

Let's create 1,000 points between -10 and 10

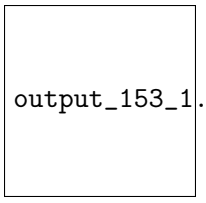
[118]:
```
x = np.linspace(-10, 10, 1000) # linspace() returns evenly-spaced numbers over a␣
 ↪specified interval
x[0:10], x[-10:]
```

[118]:
```
(array([-10.        ,  -9.97997998,  -9.95995996,  -9.93993994,
         -9.91991992,  -9.8998999 ,  -9.87987988,  -9.85985986,
         -9.83983984,  -9.81981982]),
 array([ 9.81981982,  9.83983984,  9.85985986,  9.87987988,  9.8998999 ,
         9.91991992,  9.93993994,  9.95995996,  9.97997998, 10.        ]))
```

Let's get the pdf of a normal distribution with a mean of 1 and standard deviation 3, and plot it using the grid points computed before:

[119]:
```
pdf_x = norm.pdf(x, 1, 3)
plt.plot(x, pdf_x)
```

[119]:
```
[<matplotlib.lines.Line2D at 0x71acd4859c30>]
```



And you can get random variables using the `rvs` function.

1.0.14 1.13 Dictionaries

A dictionary is another data structure (aka storage container) – arguably the most powerful. Like a list, a dictionary is a sequence of items. Unlike a list, a dictionary is unordered and its items are accessed with keys and not integer positions.

Dictionaries are the closest data structure we have to a database.

Let's make a dictionary with a few courses and their corresponding enrollment numbers.

[120]:
```
enroll2017_dict = {'CS50': 692,
                   'CS109A / Stat 121A / AC 209A': 352,
                   'Econ1011a': 95,
                   'AM21a': 153,
                   'Stat110': 485}
enroll2017_dict
```

[120]:
```
{'CS50': 692,
 'CS109A / Stat 121A / AC 209A': 352,
 'Econ1011a': 95,
 'AM21a': 153,
 'Stat110': 485}
```

One can obtain the value corresponding to a key via:

[121]:
```
enroll2017_dict['CS50']
```

[121]:
```
692
```

If you try to access a key that isn't present, your code will yield an error:

[122]:
```
# supposed to error out - READ THE ERROR MESSAGE!
enroll2017_dict['CS630']
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[122], line 2
      1 # supposed to error out - READ THE ERROR MESSAGE!
----> 2 enroll2017_dict['CS630']

KeyError: 'CS630'
```

Alternatively, the `.get()` function allows one to gracefully handle these situations by providing a default value if the key isn't found:

[123]:
```
enroll2017_dict.get('CS630', 5)
```

[123]:
```
5
```

Note, this does not *store* a new value for the key; it only provides a value to return if the key isn't found.

[124]:
```
# supposed to error out - READ THE ERROR MESSAGE!!!
enroll2017_dict['CS630']
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[124], line 2
      1 # supposed to error out - READ THE ERROR MESSAGE!!!
----> 2 enroll2017_dict['CS630']

KeyError: 'CS630'
```

[125]:
```
enroll2017_dict.get('C730', None)
```

All sorts of iterations are supported:

[126]:
```
enroll2017_dict.values()
```

[126]:
```
dict_values([692, 352, 95, 153, 485])
```

[127]:
```
enroll2017_dict.items()
```

[127]:
```
dict_items([('CS50', 692), ('CS109A / Stat 121A / AC 209A', 352), ('Econ1011a',
95), ('AM21a', 153), ('Stat110', 485)])
```

We can iterate over the tuples obtained above:

[128]:
```
for key, value in enroll2017_dict.items():
    print("%s: %d" %(key, value))
```

```
CS50: 692
CS109A / Stat 121A / AC 209A: 352
Econ1011a: 95
AM21a: 153
Stat110: 485
```

Simply iterating over a dictionary gives us the keys. This is useful when we want to do something with each item:

[129]:
```
second_dict={}

for key in enroll2017_dict:
    second_dict[key] = enroll2017_dict[key]

second_dict
```

[129]:
```
{'CS50': 692,
 'CS109A / Stat 121A / AC 209A': 352,
 'Econ1011a': 95,
 'AM21a': 153,
 'Stat110': 485}
```

The above is an actual **copy** of *enroll2017_dict's* allocated memory, unlike, `second_dict = enroll2017_dict` which would have made both variables label the same memory location.

In the previous dictionary example, the keys were strings corresponding to course names. Keys don't have to be strings, though; they can be other *immutable* data type such as numbers or tuples (not lists, as lists are mutable).

1.0.15 Dictionary comprehension: "Do not try this at home"

You can construct dictionaries using a *dictionary comprehension*, which is similar to a list comprehension. Notice the brackets {} and the use of `zip` (see next cell for more on `zip`)

[130]:
```
float_list = [1., 3., 5., 4., 2.]
int_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

my_dict = {k:v for (k, v) in zip(int_list, float_list)}
my_dict
```

[130]:
```
{1: 1.0, 2: 3.0, 3: 5.0, 4: 4.0, 5: 2.0}
```

Creating tuples with `zip` `zip` is a Python built-in function that returns an iterator that aggregates elements from each of the iterables. This is an iterator of tuples, where the i-th tuple contains the i-th element from each of the argument sequences or iterables. The iterator stops when the shortest input iterable is exhausted. The `set()` built-in function returns a `set` object, optionally with elements taken from another iterable. By using `set()` you can make `zip` printable. In the example below, the iterables are the two lists, `float_list` and `int_list`. We can have more than two iterables.

[131]:
```
float_list = [1., 3., 5., 4., 2.]
int_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

viz_zip = set(zip(int_list, float_list))
viz_zip
```

[131]:
```
{(1, 1.0), (2, 3.0), (3, 5.0), (4, 4.0), (5, 2.0)}
```

[132]:
```
type(viz_zip)
```

[132]:
```
set
```

[133]:
```
set((1,2,3,2,1,4))
```

[133]:
```
{1, 2, 3, 4}
```

1.0.16 References

A useful book by Jake Vanderplas: PythonDataScienceHandbook.

You may also benefit from using Chris Albon's web site as a reference. It contains lots of useful information.

2 END

[ ]:

[ ]:

[ ]:

[ ]: